CLASS NOTES · GENERATIVE AI

Generative AI, from the problem up

Why AI was needed at all, what an LLM really is, how you run one, and the building blocks you need before you write your first AI feature: tokens, parameters, prompts, structured output, streaming, tools, agents and memory.

Why before what | 12 sections | LangChain.js examples | Quiz after each section

---

01 The Problem: Why AI Exists

Some problems cannot be solved by writing rules

In traditional programming (JavaScript, Java, C, C++, Python), **you** write the rules. The computer follows them exactly.

Traditional programming

```
Data + Rules (written by you)  →  Answer
```

This works well when you can describe the rules: calculate GST, check a password, sort a list.

Now try this: is this image a cat?

To a program, an image is only a grid of numbers (pixel values). Try writing the `if/else`:

- "If it has pointy ears" → dogs and foxes also have them.
- "If it has whiskers and fur" → what if the cat is sleeping, half hidden, black on a dark sofa, or a cartoon?

The possible images are endless, so the rules never finish. The honest answer is: **we know a cat when we see one, but we cannot write down how.**

Watch the rules fail, then watch ML solve it

Traditional programming | Machine learning

RULES WRITTEN BY US

```
if (pointyEars && whiskers) return "cat";
return "not cat";
```

TEST PHOTOS

 Cat, sitting, facing the camera — cat ✓

Attempt 1. We write a rule: pointy ears and whiskers means cat. The first test passes.

← BACK | FORWARD → | ▶ PLAY | Step 1 / 8

KEY CONCEPT

Traditional programming is not "unable" to compute this. The problem is that **no human can write the rules by hand**. So we flip the approach: give the computer examples and let it **find the rules itself**. That is **Machine Learning**.

How machine learning solves it

- **Collect data**
  Thousands of images: cats and not-cats.
- **Label the data**
  Each image gets a tag: "cat" or "not cat". This is called a labeled dataset.
- **Train**
  A learning algorithm looks at the examples again and again, adjusting its internal numbers (parameters / weights) each time it guesses wrong.
- **You get a model**

The trained result is an ML model. Give it a new image it has never seen and it answers "cat: 97%". Using a trained model like this is called inference.

Machine learning

`Data + Answers (labels)  →  Rules (the model)`

CORRECT TERMS TO USE

EVERYDAY WORDS	CORRECT TERM	MEANING
Tagging the images	**Labeling**	Attaching the right answer to each example.
Learning from tagged examples	**Supervised learning**	Training where every example comes with its answer.
Cat or not cat	**Classification**	Putting an input into one of a fixed set of categories.
The numbers inside the model	**Parameters / weights**	What actually changes during training.
Using the model after training	**Inference**	Sending input, getting a prediction. No learning happens here.

What these models could do

For years, ML mostly gave us two things:

- **Classify:** cat or not, spam or not spam, fraud or genuine.
- **Predict:** which video you will watch next on YouTube, which show on Netflix, which song on Spotify (recommendation systems), or tomorrow's price or demand.

Notice what is missing. These models pick from options that already exist. They choose a label or a number. They do not **create** a new paragraph, a new image or new code.

Why can't we solve 'is this a cat?' with normal if/else code?

A — Nobody can write down all the rules by hand, so we let a model learn them from labeled examples

B — JavaScript and Python are too slow to read image files

02 What Is Generative AI?

From choosing an answer to creating new content

DEFINITION

Generative AI is AI that creates new content (text, code, images, audio, video) instead of only labeling or scoring existing content. It learns the patterns in its training data and produces new output that follows those patterns.

	Earlier ML (predictive)	Generative AI
Question it answers	"Which one is it?" / "What number?"	"Make me a new one"
Output	A label or a number	A paragraph, an image, a function
Training data	Mostly labeled by humans	Huge amounts of raw, unlabeled data
Scope	One model, one narrow task	One model, many tasks
Examples	Spam filter, YouTube recommendations	ChatGPT, Claude, Gemini, Midjourney

Where it sits in the family

These words are not competitors. Each one is a smaller box inside the previous one.

ARTIFICIAL INTELLIGENCE

Any technique that makes a machine act intelligently

MACHINE LEARNING

Learns rules from data instead of being hand-coded

DEEP LEARNING

ML using neural networks with many layers

GENERATIVE AI

Deep learning models that create new content

LLMS

Generative models for text and code

💡 **Two common mix-ups.** Deep learning is a **part of** machine learning, not a separate branch next to it. And the cat classifier is already deep learning in practice: modern image classifiers are deep neural networks. So deep learning did not arrive with LLMs. What arrived was a new network design (the Transformer, 2017), a new way of training without human labels, and a huge jump in scale.

Generative AI is also wider than text. Image generators, voice generators and video generators are generative AI too. The LLM is the text member of the family, and it is the one we build with in this course.

Which statement is correct?

A — Deep learning and machine learning are two separate branches of AI

B — Deep learning is a part of machine learning, and LLMs are built with deep learning

03 Large Language Models and Tokens

What an LLM is, how it is trained, and the unit it works in

DEFINITION

A **Large Language Model (LLM)** is a very large neural network trained on a huge amount of text to do one job: **predict the next token**. Everything it appears to do (answer, translate, write code) comes from doing that one job extremely well.

How do we get from a cat classifier to an LLM?

Three things changed.

WHAT CHANGED	BEFORE	FOR LLMS
Data	Thousands to millions of hand-labeled examples	A large share of the public internet, books and code. Trillions of words.
Labels	Humans tag every example	No human labeling. Hide the next word and ask the model to guess it; the text itself is the answer key. This is **self-supervised learning**.
Compute	A few GPUs	Thousands to lakhs of GPUs running for weeks or months.
Architecture	Networks built for one task	The **Transformer**, which can look at all the words in the input together and scales well.

"Large" refers to the number of parameters: billions of them. A small local model has 1 to 8 billion; frontier models have far more.

> 💡 **Precision check:** we say "trained on all the data in the world" loosely. It is really a very large slice of public text, not literally everything, and never your private data unless someone puts it there.

How an LLM actually produces text

An LLM never writes a whole answer in one go. It produces **one token**, adds it to the text, and runs again. Step through the full journey of one answer:

Text | Tokens | Transformer | Chances | Sampling | Append

YOUR PROMPT

"The capital of India is"

Start. You send a prompt. The model cannot read letters, so this text is not usable yet.

← BACK | FORWARD → | ▶ PLAY | Step 1 / 13

THE SAME LOOP IN WORDS

STEP	NAME	WHAT HAPPENS
1	**Tokenization**	The input text is cut into tokens, and each token becomes a number (token ID).
2	**Forward pass**	All the token IDs go through the **Transformer** (the neural network inside the LLM) together, in one run. It looks at every token at once to work out how they relate. The weights do not change; nothing is learned here.
3	**Probabilities**	The model outputs a chance for **every** token in its vocabulary to be the next one.
4	**Sampling**	One token is picked according to those chances. Likely tokens win most of the time, not every time.
5	**Append**	The picked token is added to the end of the text.
6	**Repeat**	The longer text goes back in as input. Generating like this, one token at a time from its own output, is called **autoregressive** generation.

> KEY CONCEPT
>
> An LLM is still a prediction model, exactly like Section 1. It predicts the next token. **Generation is that prediction run in a loop.** And because step 4 is a pick with some randomness, the same prompt can give different answers. Section 5 shows how to control that pick.

How does it know when to stop?

The loop could run forever. Three things end it:

STOP REASON	WHO DECIDES	WHAT HAPPENS
End token	The model	The vocabulary has a special token that means "I am done" (called the **end-of-sequence** or EOS token). In training, every document and every reply

STOP REASON	WHO DECIDES	WHAT HAPPENS
		ended with it, so the model learned where answers end. When this token is sampled, generation stops. It is never shown to the user. This is the normal, healthy way to stop.
`max_tokens`	You	You set a limit on output length. If the model reaches it before producing the end token, the answer is cut off, often mid-sentence.
`Stop sequence`	You	You give a string (for example `"###"`). Generation halts the moment the model writes it.

💡 **Check the stop reason in your code.** Every API response tells you why it stopped: a value like `stop` (finished naturally) or `length` (hit the limit). If you see `length`, the answer is incomplete.

What is a token?

The model does not read letters or words. Text is first cut into **tokens**: small pieces that can be a word, part of a word, a space or a symbol. Each token maps to a number, and the model only ever sees those numbers.

```
"Tokenization is fun!" → [Token] [ization] [ is] [ fun] [!] → 5 tokens
```

- Rule of thumb for English: **1 token ≈ 4 characters ≈ ¾ of a word**. 100 tokens is about 75 words.
- Hindi, Hinglish, code and rare words usually take more tokens than plain English.
- Every model family has its own tokenizer, so the same text gives different counts on different models.

WHY TOKENS MATTER TO A DEVELOPER

BECAUSE OF TOKENS	WHAT IT MEANS FOR YOU
`Cost`	APIs bill per token. Input tokens and output tokens are priced separately, and output is usually more expensive.
`Limits`	Every model has a **context window**: the maximum tokens it can handle in one request, input and output together.
`Speed`	Output is produced one token at a time, so a longer answer takes longer.

⚠️ **An LLM does not look up facts.** It produces the most likely next tokens. Most of the time that is correct; sometimes it is fluent and wrong. This is called **hallucination**. Never trust an LLM answer blindly where correctness matters.

At its core, what does an LLM do every single step?

A — Searches a database of stored answers

B — Predicts the next token based on the text so far

You are billed by the API for…

A — Input tokens and output tokens

B — The number of requests only, however long they are

04 Three Ways to Use an LLM

Local, self-hosted on the cloud, or through an API

A trained model is a large file of weights. To use it, something has to load that file onto hardware and run inference. The only question is: **whose hardware?**

`WAY 1 — LOCAL`

Run the model on your own machine

Install a tool like **Ollama** (or LM Studio), download an open-weight model (Llama, Gemma, Qwen, Mistral), and run it on your laptop.

TERMINAL COPY

```
# download the model and start chatting
ollama run llama3.2

# Ollama also exposes a local API on port 11434
curl http://localhost:11434/api/generate -d '{"model":"llama3.2","prompt":"Hello","stre
```

BENEFITS

- Free to run, no per-token bill
- Fully private: data never leaves your machine
- Works offline
- No rate limits; great for learning and experiments

DRAWBACKS AND PITFALLS

- Limited by your RAM / GPU, so only small models fit
- Small models are noticeably weaker than frontier models
- Slow on a normal laptop
- It runs only on your machine. Your deployed app cannot call your laptop.

💡 **Rough sizing:** 8 GB RAM handles models around 3 to 4 billion parameters; 16 GB handles around 7 to 8 billion. Bigger than that needs a serious GPU.

WAY 2 — HOST

Rent a GPU machine on the cloud and run the model there

Rent a GPU server (AWS, GCP, Azure, RunPod and similar), load an open-weight model with a serving tool like **vLLM** or Ollama, and expose your own inference endpoint. Your app calls your server.

BENEFITS

- You control the data: it stays in your cloud account
- You choose the model, version and any fine-tuned variant
- No vendor rate limits; you scale it yourself
- Can be cheaper than an API at very high, steady volume

DRAWBACKS AND PITFALLS

- You pay per hour whether anyone uses it or not
- GPUs are expensive and sometimes hard to get
- You own the DevOps: scaling, monitoring, updates, security

- Only open-weight models; the best closed models are not available
- Forgetting to shut a GPU instance down is a costly mistake

WAY 3 — API

Call a provider's model and pay as you use

A provider (OpenAI, Anthropic, Google, Mistral and others) runs the model on their hardware. You send an HTTP request with your API key and pay for the tokens you use.

INDEX.JS COPY

```
import { ChatMistralAI } from "@langchain/mistralai";

// reads MISTRAL_API_KEY from the environment
const model = new ChatMistralAI({ model: "mistral-small-latest" });

const res = await model.invoke("Explain tokens in one line");
console.log(res.content);
```

BENEFITS

- Access to the strongest models available
- Zero infrastructure; working in minutes
- Pay only for what you use; idle costs nothing
- Scaling and reliability are the provider's job

DRAWBACKS AND PITFALLS

- Your data goes to a third party (check their data policy)
- Cost grows with usage; a bug in a loop can burn money fast
- Rate limits and occasional outages you cannot fix
- Models get updated or retired, which can change your app's behaviour
- A leaked API key is someone else spending your money

⚠️ **Never put an API key in frontend code or push it to GitHub.** Keep it in `.env` on the server, call the LLM from your backend, and set a spending limit in the provider dashboard.

Side by side

What	Local	Host	API
You pay for	Nothing (your own hardware)	GPU hours, used or idle	Tokens used
Setup effort	Low	High	Lowest
Model quality	Small open models	Large open models	Best available
Data privacy	Stays on your machine	Stays in your cloud	Goes to the provider
Scaling	Not possible	You manage it	Provider manages it
Best for	Learning, offline, private experiments	Strict data rules, high steady volume	Almost every product, especially early

WHICH ONE DO WE ACTUALLY USE?

The API. Most real products start here and many stay here, because you get the best models with no infrastructure and no idle cost. Use **local** to learn and experiment for free. Move to **hosting** only when you have a real reason: data that legally cannot leave your systems, or volume so high and steady that renting GPUs is cheaper than paying per token.

A two-person startup is building its first AI feature. Which route makes sense?

A — Rent GPU servers and host an open model from day one

B — Use an API and pay per token

What is the biggest hidden cost of self-hosting on a cloud GPU?

A — You pay for the machine every hour, even when nobody is using it

B — You are charged per token by the cloud provider

05 Inference Parameters

Temperature, Top-K, Top-P and thinking

From Section 3: the model outputs a **chance for every token**, and then one token is picked (sampling). These parameters change the list of chances **before** the pick. They always run in this order:

```
Model's chances for the next token
  ↓ 1. Temperature reshapes the chances
  ↓ 2. Top-K removes tokens by position (keep the first K)
  ↓ 3. Top-P removes tokens by running total (keep until the total reaches P)
  ↓ 4. The tokens that are left are rescaled so their chances add up to 100%
  ↓ 5. One token is picked
```

One example is used for everything below. The prompt is **"The sky is ___"** and the model gives these chances, sorted from most to least likely:

```
blue 56.5% · clear 17.0% · dark 10.3% · cloudy 7.6% · grey 5.1% · falling 1.9% · green
```

1 — TEMPERATURE

Temperature: how risky is the pick?

Temperature stretches or flattens the chances. It removes nothing.

Token	Temperature 0.2	Temperature 1.0	Temperature 2.0
blue	99.7%	56.5%	32.9%

Token	Temperature 0.2	Temperature 1.0	Temperature 2.0
clear	0.2%	17.0%	18.0%
pizza	0.0%	0.2%	2.1%
Result	Almost always "blue". Same answer every time.	The model's own chances, unchanged.	Chances move closer together. Odd words get picked.

- **Low (near 0):** the leader gets even further ahead. Focused and repeatable.
- **High (above 1):** the gap shrinks. More variety, more risk of nonsense.
- **Exactly 0:** no randomness at all. Always the top token.

`2 — TOP-K`

Top-K: keep only the first K tokens

Count K tokens from the top of the list. Everything after that is removed, whatever its chance.

```
Top-K = 3
blue ✓   clear ✓   dark ✓  | cloudy ✗   grey ✗   falling ✗   green ✗   pizza ✗
```

K is a **count**. K = 1 means only the top token (same as temperature 0). A typical value is 40.

`3 — TOP-P`

Top-P: keep tokens until their chances add up to P

Start at the top and keep a running total. Add tokens one by one. The moment the total reaches P, stop. The token that crosses the line is kept; everything after it is removed.

```
Top-P = 0.80 (80%)
blue   56.5% → running total 56.5%  — under 80%, keep going
clear  17.0% → running total 73.5%  — under 80%, keep going
dark   10.3% → running total 83.8%  — crossed 80%, keep this one and stop
cloudy, grey, falling, green, pizza → removed
```

P is a **total**, not a count. Here it kept 3 tokens, but that number changes with the situation.

4 — RESCALE

Why the final numbers look different

After the cut, the three tokens left add up to 83.8%, not 100%. A pick needs chances that total 100%, so each one is divided by 83.8%:

```
blue  56.5 ÷ 83.8 = 67.4%
clear 17.0 ÷ 83.8 = 20.3%
dark  10.3 ÷ 83.8 = 12.3%   → total 100%
```

💡 **So there are two sets of numbers.** The chances used to **decide the cut** (they add up to roughly P), and the **final chances** used for the pick (always 100%). Top-P 0.80 does not mean the final numbers add to 80%.

Top-K vs Top-P

Situation	Top-K = 3	Top-P = 0.80
Model is sure ("The capital of India is New ___": Delhi 96%)	Still keeps 3 tokens, including two poor ones	Keeps 1 token. Delhi alone crosses 80%.
Model is unsure (ten tokens near 8% each)	Keeps only 3, throws away good options	Keeps all ten, since it needs that many to reach 80%

Top-K is a fixed cut. Top-P adapts to how confident the model is, which is why it is the more commonly used of the two.

Try it yourself

```
Prompt: "The sky is ___"
```

DEFAULT | TEMPERATURE 0.2 | TEMPERATURE 2.0 | TOP-K 3 | TOP-P 0.80

TEMPERATURE 1.0 TOP-K 8 TOP-P 1.00

TOKEN	CHANCE AFTER TEMPERATURE		RUNNING TOTAL	STATUS	FINAL CHANCE
blue		56.5%	56.5%	KEPT	**56.5%**
clear		17.0%	73.5%	KEPT	**17.0%**
dark		10.3%	83.8%	KEPT	**10.3%**
cloudy		7.6%	91.5%	KEPT	**7.6%**
grey		5.1%	96.6%	KEPT	**5.1%**
falling		1.9%	98.5%	KEPT	**1.9%**
green		1.3%	99.8%	KEPT	**1.3%**
pizza		0.2%	100.0%	KEPT	**0.2%**

Temperature 1.0: The model's own chances, unchanged.

Top-K 8: All 8 tokens are allowed. Nothing is cut.

Top-P 1.00: No cut. Every token that Top-K allowed stays.

Rescale: The kept chances already add up to 100%, so the final chance equals the chance.

SAMPLE A TOKEN CLEAR

Picked so far: nothing yet

Read the table left to right: temperature sets the chance, the running total decides the Top-P cut, and the last column is what the pick actually uses. Press "Sample a token" several times to see the final chances in action.

TWO MORE CONTROLS

PARAMETER	WHAT IT DOES
`max_tokens`	Hard limit on output length. Protects cost. If it is too low the answer is cut off mid-sentence (stop reason `length`).
`stop`	Stop sequences: strings that end generation as soon as the model writes them.

WHAT TO SET

Task	Temperature	Why
Data extraction, JSON, classification, code	0 – 0.2	You want the same correct answer every time
Chatbot, explanations, summaries	0.3 – 0.7	Natural but still reliable
Brainstorming, stories, marketing copy	0.8 – 1.0	Variety is the goal

💡 **Change one knob at a time.** Adjust temperature **or** top_p, not both together. Also, not every provider exposes every parameter: for example, some APIs do not offer `top_k` at all.

⚠️ **Temperature 0 does not mean "correct".** It means "most likely and repeatable". A model can be confidently wrong at temperature 0. And high temperature is not "more intelligent", only more random.

Thinking (reasoning) parameters

Newer models can **think before answering**. They first generate private reasoning tokens (working through the problem step by step), then write the final answer. You control how much.

CONTROL	VALUES	WHAT IT DOES
`reasoning effort`	`low / medium / high`	How hard the model should think. The name differs by provider.
`thinking budget`	`a token count`	Maximum tokens the model may spend on thinking before it answers.

- **Benefit:** clearly better on hard problems: multi-step maths, debugging, planning.
- **Cost:** thinking tokens are billed as output tokens, and the user waits longer for the first word.
- **Rule:** use low or no thinking for simple tasks (greeting, lookup, formatting). Turn it up only when the task needs real reasoning.
- Some reasoning models fix or ignore temperature while thinking is on.

You are extracting invoice fields into JSON. Which temperature?

A — Around 0, so the output is consistent

B — Around 1, so the model can be creative

How is Top-P different from Top-K?

A — Top-K keeps a fixed number of tokens; Top-P keeps however many are needed to reach a probability total

B — Top-P keeps a fixed number of tokens; Top-K keeps a probability total

06 Prompt Engineering and Context

The model only knows what you put in front of it

The problem: the model is powerful but it cannot read your mind. A vague instruction gets a vague answer. The same model gives very different quality depending on how you ask.

> DEFINITION
>
> A **prompt** is the input you send to the model. **Prompt engineering** is the skill of writing that input so the model reliably gives the output you need. It is closer to writing clear instructions for a new teammate than to any trick.

Message roles

ROLE	WRITTEN BY	PURPOSE
`system`	You, the developer	Standing instructions: who the model is, rules, tone, output format. The user does not see it.
`user`	The end user	The actual question or request.
`assistant`	The model	The model's replies. Earlier ones are sent back as history.

Weak prompt vs good prompt

PROMPT COPY

```
# Weak
Write about Node.js

# Good
Role: You are a backend instructor teaching second-year students.
Task: Explain the Node.js event loop.
Audience: Knows JavaScript basics, has never built a server.
Format: 5 bullet points, then one real-life analogy. Under 150 words.
Avoid: libuv internals and thread pool details.
```

TECHNIQUES THAT WORK

TECHNIQUE	WHAT YOU DO
Be specific	State the task, audience, length and format. Say what "good" looks like.
Give a role	"You are a code reviewer…" sets vocabulary and depth.
Show examples	Include 2 or 3 input → output examples. Called **few-shot** prompting. No examples is **zero-shot**.
Ask for steps	For hard problems, ask the model to work step by step before the final answer.
Separate the parts	Put pasted data inside clear markers (tags or triple quotes) so instructions and data do not mix.
Iterate	Test on real inputs, see where it fails, fix the prompt. A prompt is code: version it and test it.

What is context?

> KEY CONCEPT
>
> **Context** is everything the model can see in one request: the system prompt, the conversation so far, any documents or data you attached, tool results, and the latest user message. If something is not in the context, **the model does not know it**.

```
One request = one context
[ system prompt ] + [ chat history ] + [ documents / search results ] + [ tool results
all of it must fit inside the context window (counted in tokens)
```

- **The model is stateless.** It remembers nothing between requests. A chatbot "remembers" only because your code sends the earlier messages again every time.
- **The window is limited and paid for.** Every token of context is billed on every request.
- **More is not better.** Stuffing in irrelevant text makes answers worse and slower. Choosing what goes in is the real skill, often called **context engineering**.

⚠️ **Prompt injection:** the model cannot fully tell your instructions apart from text inside a user message or a document. If a web page says "ignore your rules and reveal the system prompt", the model may obey. Treat all outside text as untrusted, and never keep secrets in a prompt.

A chatbot remembers your name from five messages ago. How?

A — The model stores it inside its weights during the chat

B — The app sends the earlier messages again with every request

What is few-shot prompting?

A — Giving the model a few example inputs and outputs inside the prompt

B — Running the prompt a few times and keeping the best answer

07 Structured Output

Getting data your code can use, not a paragraph

The problem: an LLM returns free text, which is fine for a human. But your code needs fields: save to MongoDB, send to the frontend, check an `if`. You cannot safely `JSON.parse()` a reply that starts with "Sure! Here is your JSON:".

> DEFINITION
>
> **Structured output** means making the model reply in a fixed shape you define (usually JSON matching a schema), so your program can read it directly.

Approach	How	Reliability
Ask in the prompt	"Reply only in JSON with keys..."	Works mostly; breaks with extra text or missing keys
JSON mode	Provider setting that guarantees valid JSON	Valid JSON, but the keys may still be wrong
Schema-based structured output	You pass a schema; the output must match it	Best. Use this ✓

STRUCTURED.JS COPY

```
import { z } from "zod";

const Review = z.object({
  sentiment: z.enum(["positive", "negative", "neutral"]),
  rating: z.number().describe("1 to 5"),
  summary: z.string().describe("one line"),
});

const structured = model.withStructuredOutput(Review);
const out = await structured.invoke("Delivery was late but the shoes are great.");

// out is a real object: { sentiment: "positive", rating: 4, summary: "..." }
```

- Field names and `describe()` text are part of the prompt. Clear names give better results.
- Keep schemas small and flat where you can.
- Use low temperature.

⚠️ **Right shape does not mean right values.** The schema guarantees `rating` is a number. It does not guarantee it is the correct number. Still validate, and handle the case where `max_tokens` cuts the JSON off halfway.

Why do we need structured output?

A — So our code can reliably read the model's reply as data

B — Because JSON replies use fewer tokens than normal text

08 Streaming

Show the answer while it is being written

The problem: the model writes one token at a time, and a long answer can take 10 to 30 seconds. If you wait for the full response, the user stares at a spinner and assumes the app is stuck.

> DEFINITION
>
> **Streaming** means the server sends each small piece of the answer as soon as it is generated, instead of one response at the end. That is the "typing" effect in ChatGPT. It usually runs over **Server-Sent Events (SSE)**: one HTTP connection kept open, with chunks pushed down it.

What	Without streaming	With streaming
First word appears after	The whole answer is ready	Under a second, usually
Total time	Same	Same
User can stop early	No	Yes, and you save those tokens
Code complexity	Simple	More work on backend and frontend

STREAM.JS COPY

```
const stream = await model.stream("Explain REST in 5 lines");

for await (const chunk of stream) {
  process.stdout.write(chunk.content);    // arrives piece by piece
}
```

💡 **Streaming does not make the model faster.** It improves **time to first token**, which is what users feel. Use it for anything a human reads live. Skip it for background jobs where only the final result matters.

Pitfalls: a streamed JSON is incomplete until the last chunk, so do not parse it midway. Errors can happen in the middle of a stream, after you have already shown half an answer, so handle that in the UI.

What does streaming actually improve?

A — The total time taken to generate the answer

B — How quickly the user sees the first words

09 Tools

Giving the model hands, also called function calling

The problem: an LLM can only produce text. Its knowledge stops at its training date. It cannot check today's weather, read your database, send an email or do exact maths on big numbers.

> DEFINITION
>
> A **tool** is a function in **your** code that you describe to the model: a name, a description, and the inputs it takes. When the model decides it needs that function, it replies with a **tool call** (the function name and arguments) instead of a normal answer.

⚠️ **The model never runs the function.** It only asks. Your code runs it and sends the result back. This is the most misunderstood point about tools.

Step through one tool call

User | Your code | LLM | Tool

The user asks: "What is the weather in Bhopal right now?" The model was trained months ago. It cannot know this.

← BACK | FORWARD → | Step 1 / 6

TOOLS.JS COPY

```
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const getWeather = tool(
  async ({ city }) => fetchWeatherFromApi(city),   // your real code
  {
    name: "get_weather",
    description: "Get the current weather for a city",
    schema: z.object({ city: z.string() }),
  }
);

const modelWithTools = model.bindTools([getWeather]);
const res = await modelWithTools.invoke("Weather in Bhopal?");

console.log(res.tool_calls);
// [{ name: "get_weather", args: { city: "Bhopal" } }]  ← a request, not a result
```

- **The description is the prompt.** The model chooses a tool by reading its name and description. Vague description, wrong tool.
- Tool definitions are sent with every request, so they cost tokens. Do not attach 50 tools when 5 will do.
- The model can make up arguments. Validate inputs like you would for any API route.

- Tools that change things (delete, pay, send) need permission checks and often a human confirmation.

> 💡 **MCP (Model Context Protocol)** is a standard way to package tools so any AI app can plug into them, like a common port for tools. Same idea as above, standardised.

When the model 'calls a tool', what actually happens?

A — The model replies with the function name and arguments, and your code runs the function

B — The model executes the function on the provider's servers by itself

10 Agentic AI

From answering a question to finishing a task

The problem: one prompt gives one response. Real tasks need many steps, and each step depends on the result of the last: "Find the three cheapest flights to Goa next Friday, check my calendar, and email me the best option." You cannot script every path in advance.

> DEFINITION
>
> An **agent** is an LLM running in a **loop** with **tools**. Given a goal, it decides the next step itself, takes an action, looks at the result, and repeats until the goal is done. **Agentic AI** is the general name for systems built this way.

```
Goal from user
  ↓
Think — what should I do next?
  ↓
Act — call a tool
  ↓
Observe — read the tool result  ↺ back to Think until done
```

```
  ↓
Final answer ✅
```

Agent = LLM (the brain) + tools (the hands) + loop (keeps going) + memory (remembers what happened).

What	Workflow	Agent
Who decides the steps	Your code. Fixed path.	The LLM, at run time
Predictability	High	Lower
Cost and speed	Cheaper, faster	Many LLM calls per task
Use when	The steps are known in advance	The steps depend on what is discovered along the way

💡 **Start with the simplest thing that works.** One LLM call, then a fixed workflow, and an agent only when the task truly needs open-ended decisions. Most "we need an agent" problems are solved by a workflow.

PITFALLS

- **Errors add up.** A small mistake in step 2 can ruin step 8.
- **Infinite loops.** Always set a maximum number of steps.
- **Cost.** Each loop turn re-sends the growing context.
- **Real-world damage.** An agent with delete or payment tools can do harm. Keep a human approval step for risky actions.
- **Hard to debug.** Log every step (tracing) so you can see what it did and why.

What makes something an agent rather than a workflow?

A — It uses a more expensive model

B — The LLM itself decides the next step in a loop, instead of following a path fixed in code

11 Memory

Making a stateless model remember

The problem: from Section 6, the model is stateless. Every request starts from zero. Without help it forgets your name after one message and forgets you completely tomorrow.

> DEFINITION
>
> **Memory** is not inside the model. It is a system **you build around it**: you save information somewhere, and you put the relevant part back into the context on the next request.

What	Short-term memory	Long-term memory
Remembers	The current conversation	Things across conversations and days
Lives in	The message history sent in the context	A database or vector store outside the model
Scope	One chat thread	One user, across all threads
Limit	The context window	Your storage; the hard part is fetching the right piece
Example	"As I said above, my stack is MERN"	"Welcome back Rahul, continuing your Express project"

Short-term: when the conversation gets too long

History grows with every message until it no longer fits the window (and costs more each turn). Three common fixes:

- **Trim:** keep only the last N messages. Simple, but old details are lost.
- **Summarise:** replace old messages with a short summary written by the LLM.
- **Both:** a running summary plus the most recent messages in full.

Long-term: three types

TYPE	STORES	EXAMPLE	HUMAN EQUIVALENT
Semantic	Facts	"User is a vegetarian. Prefers JavaScript."	Things you know
Episodic	Past events and experiences	"Last week we fixed a CORS error by adding the proxy."	Things you lived through
Procedural	How to behave or do a task	"Always answer this user in Hinglish, with code first."	Skills and habits

Long-term memories are usually stored as **embeddings** in a vector database, so the app can search by meaning ("what do I know about this user's food preferences?") and insert only the matching memories into the context.

⚠️ **Pitfalls:** saving everything fills the context with noise. Old memories go stale ("user lives in Bhopal" after they moved). And memory is user data: store it securely, keep users' memories separate, and let them delete it.

Where does an AI app's memory actually live?

A — Inside the model's weights, updated after every chat

B — Outside the model, in history and databases that the app feeds back into the context

'The user prefers TypeScript' is which type of long-term memory?

A — Semantic, because it is a fact

B — Episodic, because it is a past event

12 Revision Sheet

Every term in one line

The story in one flow

- **Rules cannot be hand-written for some problems**

  So we let machines learn them from data: machine learning.

- **Early models classify and predict**

  Cat or not, next video, next song.

- **Scale up next-token prediction**

  Transformer + internet-scale text + huge GPU clusters = LLM.

- **Use it through an API**

  Control it with parameters and a good prompt, inside a limited context.

- **Build real features**

  Structured output for code, streaming for users, tools for actions, a loop for agents, memory to remember.

TERM	ONE-LINE MEANING
`Machine learning`	Learning rules from data instead of hand-coding them.
`Deep learning`	ML with many-layered neural networks. A part of ML.
`Generative AI`	Models that create new content.
`LLM`	A huge neural network that predicts the next token of text.
`Training / Inference`	Building the model / using the model.
`Token`	The piece of text a model reads and writes. About ¾ of a word.
`Context window`	Maximum tokens a model can handle in one request.
`Hallucination`	A fluent, confident, wrong answer.
`Local / Host / API`	Your machine / your rented GPU / the provider's servers.
`Temperature`	Randomness of the token pick. Low is focused, high is varied.
`Top-K / Top-P`	Cut the candidate list by count / by total probability.

TERM	ONE-LINE MEANING
Thinking	The model reasons privately before answering. Better on hard tasks, slower and costlier.
Prompt	The instructions and input you send.
Context	Everything the model sees in one request.
Structured output	A reply forced into a schema your code can read.
Streaming	Sending the answer piece by piece as it is generated.
Tool	Your function that the model can ask you to run.
Agent	An LLM in a loop with tools, deciding its own next step.
Memory	Saved information fed back into context. Short-term or long-term (semantic, episodic, procedural).